Autonomous Line-Following Robot with Load Balancing

Critical Design Review — Final Report

Team 3D: Titus Tito, Victor Lee, Arturo Escobar, John Whitmarsh **Course:** ENGR 100L, Section 003

TA: Sumia Saleh **Date:** December 2, 2025

1. Objective

This report explains the design, construction, and testing of an autonomous robot built for a line-following competition with a load-balancing requirement. The robot had to follow a marked path across a course of straight sections and tight corners while carrying an unsecured 250 g load on a raised platform. It also had to finish the course as fast as possible without dropping the load. To meet this goal, our team worked across four areas: the mechanical design of the chassis and load platform, the electrical design of the sensors and power system, the programming that controlled line-tracking and speed, and structured testing to tune and check performance.

2. Materials and Setup

We built the robot using only the parts provided for the competition:

- Arduino Uno microcontroller
- Breadboard
- Two TT gear motors and wheels, with one trailing caster
- L298N dual H-bridge motor driver
- Two IR reflectance sensors
- Buzzer, LCD display, LEDs, resistors
- AA battery pack (8 cells)
- 3D-printed chassis and custom mounts, designed in Fusion 360
- Screws, bolts, and zip ties for assembly and cable management

We designed the chassis and all the custom mounts in Fusion 360, then 3D printed every part. We kept the robot's size under 250 mm long, 180 mm wide, and 160 mm high. The load platform is round, 60 mm across, and can hold up to 250 g without extra support. We placed the electronics and battery pack where they were easy to reach for testing and repairs, and we kept the wiring short and tied down so it would not tangle or block the sensors. We built, wired, and tested everything in the UM-Dearborn engineering lab.

3. Mechanical Design

We modeled the chassis fully in Fusion 360 before printing any parts. This let us check fit and accessibility from multiple angles before fabrication. Figures 1 and 2 show the front and side views we used to confirm platform symmetry, sensor placement, wheelbase, and battery position. Figure 3 shows the angled view we used for a final check of part clearance and overall compactness.

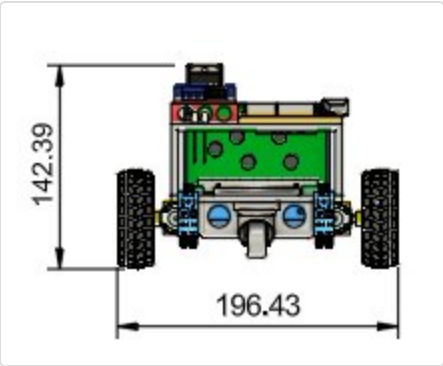


Figure 1. Fusion 360 front view, showing platform symmetry and sensor placement.

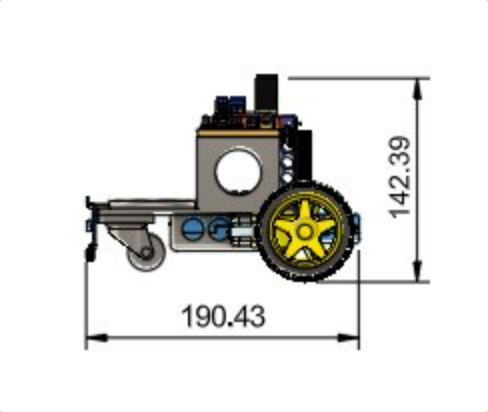


Figure 2. Fusion 360 side view, showing chassis height, wheelbase, and battery position.

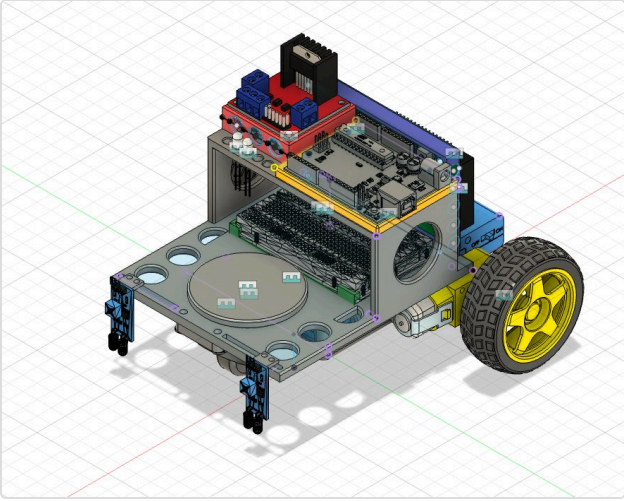


Figure 3. Fusion 360 angled view, used to verify final part arrangement and clearance.

Keeping the load platform centered over the wheelbase was the main goal through several design rounds. An off-center platform made the load more likely to tip during turns. After printing and finishing the parts, we assembled the complete chassis with all the electronics and wiring installed, shown in Figure 4.

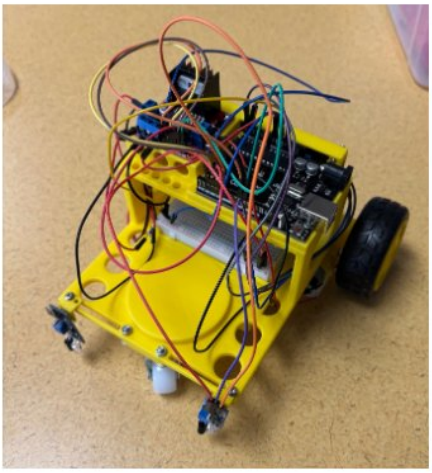


Figure 4. Completed 3D-printed chassis with all components and wiring installed.

4. Circuit Design

We made sure the wiring was safe, clear, and easy to repair. Table 1 shows the pin assignments we used across the build.

Function	Arduino Pin(s)
Motor direction control	2, 3, 4, 5
Motor speed (PWM)	9, 10
IR line/marker sensors	6, 7
LEDs and buzzer (status feedback)	A2, A3, 8
LCD (speed and diagnostics display)	Standard LCD interface pins

We split power between the logic circuit and the motors to keep sensor readings stable under motor load. Figure 5 shows the full wiring diagram with pinouts and component locations.

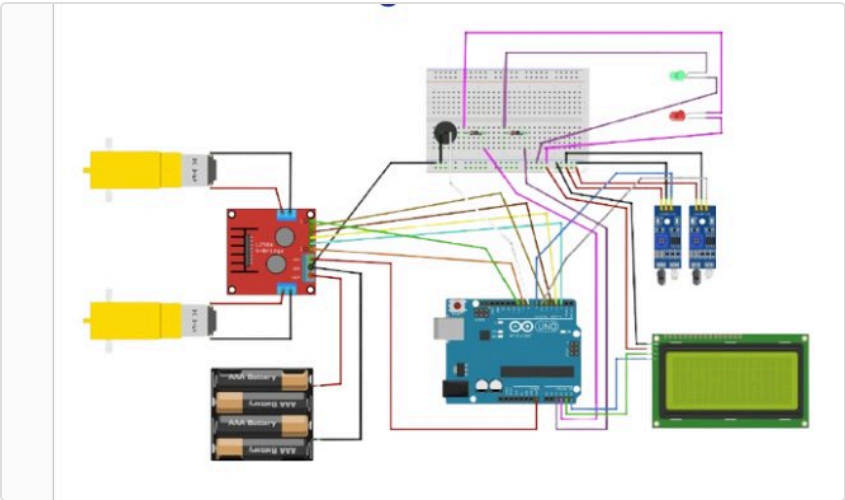


Figure 5. Circuit wiring diagram with pinouts and component locations.

We checked every connection with a multimeter before powering the robot on, to make sure there were no loose wires or shorts that could interfere with how it ran.

5. Control System Design

Our Arduino program controlled speed, turning, and obstacle response through a small set of tunable settings: a *targetVelocity* for straight-line travel, a slower *turnVelocity* for cornering, and a *collisionVelocity* for when the robot detected an obstacle or marker. We smoothed the transitions between these settings with gradual acceleration and deceleration instead of instant speed changes. This reduced the sudden weight shifts that could tip the load. The two IR sensors constantly checked the position of the line and flagged corner transitions in real time, and the onboard LEDs, buzzer, and LCD gave us live feedback for debugging during test runs.

6. Test Methodology

We tested the robot repeatedly on a course with straight segments and tight corners, changing one setting at a time so we could isolate its effect. We used fresh batteries for every run to keep power delivery consistent across tests. On every run, we tracked five things: completion time, obstacle avoidance, load retention, vehicle speed, and line-tracking accuracy.

7. Data Analysis

We analyzed the results from ten test runs in MATLAB, where we built heatmaps comparing our settings to the outcomes. This let us see at a glance which speed and turning-rate combinations tended to drop the load and which produced the fastest, most stable runs, instead of comparing raw numbers row by row.

8. Results

Early runs at higher speeds and sharper turning rates caused the load to drop on some corners. Mid-run adjustments to speed and turn-in behavior steadily reduced these drops. By the final runs, tuned to an operating

speed of 0.08–0.15 m/s with softened acceleration into turns, the robot completed the course in under 60 seconds with zero load drops.

Test Phase	Operating Speed	Load Retention	Completion Time
Early runs	Higher speed, sharp turns	Intermittent drops	Inconsistent
Mid runs	Reduced speed, softened turns	Improved, occasional drops	Approaching 60 s
Final runs	0.08–0.15 m/s	Zero drops	Under 60 s

9. Discussion

Our first full assembly was larger than it needed to be, and the load tipped often during turns. To fix this, we adjusted the sensor placement, lowered the battery position to bring down the center of gravity, and reshaped the acceleration curve in the control code. We tested each change on its own so we could measure its effect directly, which kept the tuning process from turning into guesswork. Splitting the work across CAD, circuits, and code let us work on all three at the same time and bring the pieces together before the deadline.

10. Conclusion

Our final robot met every requirement for footprint, parts list, power budget, assembly quality, and platform stability, while staying modular enough for quick adjustment during testing. It tracked the line reliably, changed speed smoothly on turns, and gave us clear real-time feedback for tuning performance. Beyond the competition result, this project walked us through the full engineering process, from CAD and circuit design through programming, structured testing, and data-driven changes.

11. Project Team

Titus Tito

Led the chassis design in Fusion 360 and contributed to assembly and testing. A first-year Human-Centered Engineering Design student from Canton, Michigan, drawn to how systems are built and to hands-on technical work.

Victor Lee

Mechanical Engineering student with hands-on fabrication experience from home remodeling and deck building since age 13. Plans to pursue a career in nuclear engineering.

Arturo Escobar

Mechanical Engineering student from Las Vegas, Nevada, with a particular interest in mathematics.

John Whitmarsh

Mechanical Engineering student from Dearborn, Michigan, with over ten years of hands-on experience in home restoration and remediation.